Are NSGA-III Implementations Really Stable? A Cross-Platform Analysis of Anomalous Behaviors of NSGA-III

Kenneth Marshall Zhang
Department of Computer Science and Engineering Southern University of Science and Technology
Shenzhen 518055, China
kennethz@mail.sustech.edu.cn

Lie Meng Pang
Department of Computer Science and Engineering Southern University of Science and Technology
Shenzhen 518055, China
panglm@sustech.edu.cn

Hisao Ishibuchi*
Department of Computer Science and Engineering Southern University of Science and Technology
Shenzhen 518055, China
hisao@sustech.edu.cn

Abstract

NSGA-III is widely used as a baseline algorithm in multi- and many-objective optimization. Since no official source code was released, researchers rely on third-party implementations, with PlatEMO and pymoo being the most frequently used. In this paper, we examine whether these implementations exhibit different search behaviors on continuous multi-objective optimization problems. Through a case-by-case analysis of 88 problems from 13 test suites (100 problem instances), we confirm clearly divergent search behaviors. We propose stability metrics based on temporal stability of the estimated nadir point to systematically compare the two implementations. Overall, pymoo's NSGA-III outperforms PlatEMO's on most problems, which we attribute to differences in their nadir point estimation mechanisms.

CCS Concepts

• **Theory of computation** → **Evolutionary algorithms**; **Continuous optimization**.

Keywords

Evolutionary multi-objective optimization, NSGA-III, normalization, nadir point estimation

ACM Reference Format:
Kenneth Marshall Zhang, Lie Meng Pang, and Hisao Ishibuchi. 2026. Are NSGA-III Implementations Really Stable? A Cross-Platform Analysis of Anomalous Behaviors of NSGA-III. In *Genetic and Evolutionary Computation Conference Companion (GECCO'26 Companion), July 13–17, 2026, San José, Costa Rica.* ACM, New York, NY, USA, 4 pages.

1 Introduction

Multi-objective evolutionary algorithms (MOEAs) evolve populations to approximate the Pareto front (PF) while balancing convergence and diversity [16]. Among MOEAs, NSGA-III [4] is frequently used as a baseline in the evolutionary multi-objective optimization (EMO) community. However, NSGA-III was released without source code, leaving only community implementations on platforms such as PlatEMO [12] and pymoo [1, 2].

For combinatorial MOPs, Gong et al. [6] reported that these implementations exhibit different search behaviors. However, no analogous findings have been reported for continuous MOPs. In this paper, we examine whether they exhibit different search behaviors across a broader range of continuous problems. Through a case-by-case analysis of 88 problems from 13 test suites, we confirm clearly divergent behaviors. Across 100 problem instances, both perform similarly on 37, pymoo performs better on 36, PlatEMO performs better on 2, and neither performs well on the remaining 25. We attribute these differences to their nadir point estimation implementations and systematically measure stability using proposed metrics.

2 Background

2.1 Overview of NSGA-III

We briefly describe NSGA-III's environmental selection [2, 4]. Given parent and offspring populations of size N, the combined population is sorted into nondominated fronts. The ideal point is estimated as $z_j^* = \min_{\mathbf{x}\in S^{(t)}} f_j(\mathbf{x})$. For the nadir point, m extreme points are selected by minimizing the Tchebycheff scalarizing function, and a hyperplane through them yields the estimated nadir point $\mathbf{z}^{\text{nad}}$. Solutions are then normalized and associated with reference vectors via perpendicular distance for niching-based selection.

2.2 Different Nadir Point Estimation

We denote the implementations as Pl-NSGA-III and Py-NSGA-III. Fig. 1 displays their workflows. The key differences arise in two edge cases. When the hyperplane equation is singular (fewer than m unique extreme points), Pl-NSGA-III falls back to the maximum objective values from surviving solutions, while Py-NSGA-III uses the maximum from the nondominated front. When estimated nadir components fall below the ideal point, Pl-NSGA-III proceeds without correction, whereas Py-NSGA-III applies a multi-stage fallback mechanism.

To confirm that nadir point estimation causes the observed differences, we created cross-platform reimplementations by swapping only the nadir estimation module. In all tested configurations, implementations sharing the same nadir estimation exhibited identical search behaviors.

2.3 Examples of Different Search Behaviors

Figs. 2 and 3 show final populations on three-objective IDTLZ2 [10] (median-HV run, 31 runs, 100,000 FE). At $N = 91$, both yield well-distributed solutions. At $N \in \{105, 120\}$, Pl-NSGA-III produces

*Corresponding author.

GECCO'26, San José, Costa Rica
© 2026 Copyright held by the owner/author(s).
This is the author's version of the work. It is posted here for your personal use. Not for redistribution. The definitive Version of Record was published in *Genetic and Evolutionary Computation Conference Companion (GECCO'26 Companion), July 13–17, 2026, San José, Costa Rica.*

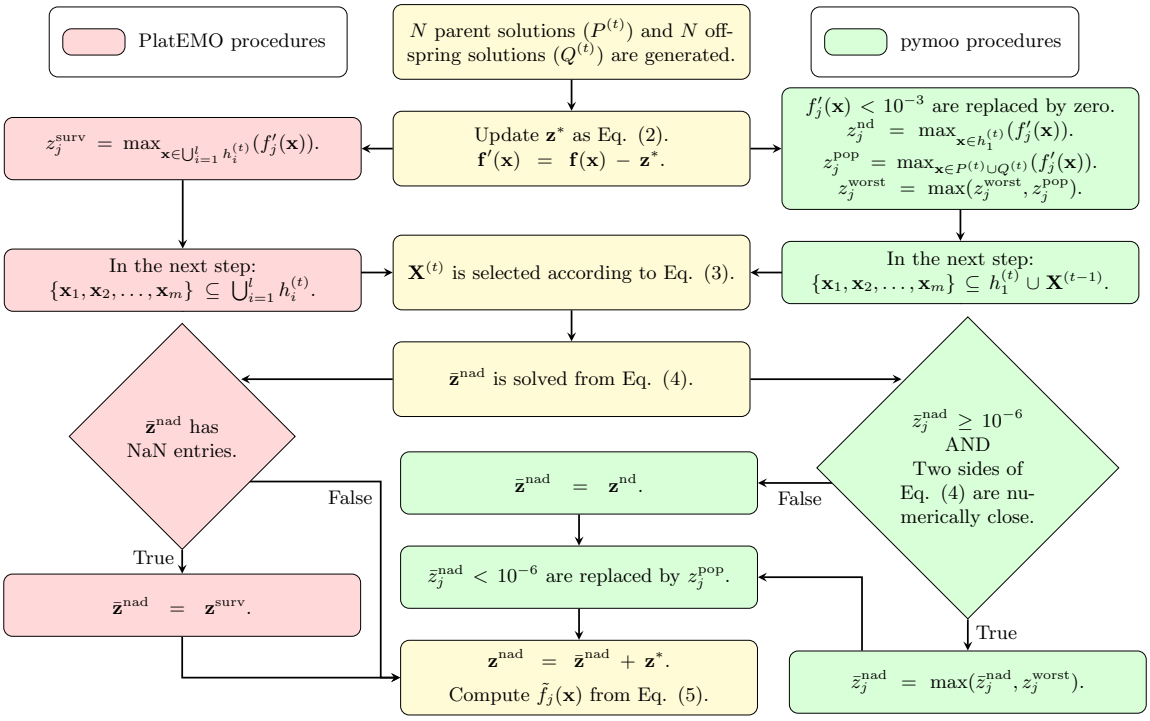
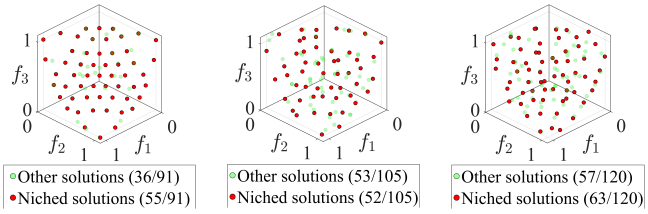
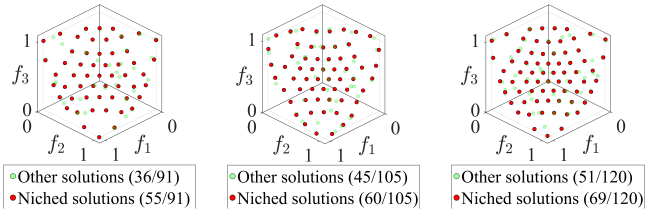


Figure 1: Normalization mechanism of NSGA-III in PlatEMO and pymoo. Yellow: common procedures; red: PlatEMO-specific; green: pymoo-specific. j ranges through $1, 2, \ldots, m$.



(a) $N = 91$. **(b)** $N = 105$. **(c)** $N = 120$.

Figure 2: Final populations of IDTLZ2 for Pl-NSGA-III.



(a) $N = 91$. **(b)** $N = 105$. **(c)** $N = 120$.

Figure 3: Final populations of IDTLZ2 for Py-NSGA-III.

poorly-distributed results while Py-NSGA-III remains stable. Similar phenomena occur on Minus-DTLZ1 and Minus-DTLZ2 [9].

3 Measuring the Stability of NSGA-III

Stable normalization requires consistent ideal/nadir point estimation. We assess stability via *temporal stability*: does the estimated nadir point converge within a run?

Given estimated nadir points $\{\mathbf{z}^{(1)}, \ldots, \mathbf{z}^{(T)}\}$ over T generations, we normalize them by the true ideal/nadir points. We compute the *tail mean* $\boldsymbol{\mu}(t) = \frac{1}{T-t+1}\sum_{k=t}^{T}\tilde{\mathbf{z}}^{(k)}$ and the *tail max-deviation* $\sigma_{\max}(t) = \max_{k\geq t}\|\tilde{\mathbf{z}}^{(k)} - \boldsymbol{\mu}(t)\|_2$. A run is **temporally stable** if either: (1) $\exists\, t,\ \sigma_{\max}(t) < 10^{-6}\cdot\sigma_{\max}(1)$, or (2) $\sigma_{\max}(1) < 10^{-3}$.

Fig. 4 illustrates temporal stability on the three-objective real-world problem RWA3 [7]. Both algorithms started with the same initial population. Py-NSGA-III achieves temporal stability at generation $t^* = 56$, whereas Pl-NSGA-III never stabilizes.

4 Experimental Results

4.1 Experimental Setup

We evaluate Pl-NSGA-III and Py-NSGA-III on 88 problems from 13 test suites: DTLZ1–7 [5], IDTLZ1–2 [10], SDTLZ1–2 [4], Minus-DTLZ1–6 [9], IMOP1–8 [13], BT1–9 [11], MaF1–5, MaF8–9, MaF13–15 [3], WFG1–9 [8], Minus-WFG1–9 [9], RWA1–7, RWA9 [7], UF1–10 [15], VNT1–3 [14], and ZDT1–4, ZDT6 [17]. We also test DTLZ1–2, IDTLZ1–2, and WFG1–2 with $m \in \{5, 10\}$ objectives, totaling 100 problem instances. Each algorithm–problem pair is run 101 times with shared initial populations. Maximum FE is 200,000. Population size N is 120 ($m \in \{2, 3\}$), 210 ($m = 5$), or 440 ($m = 10$).

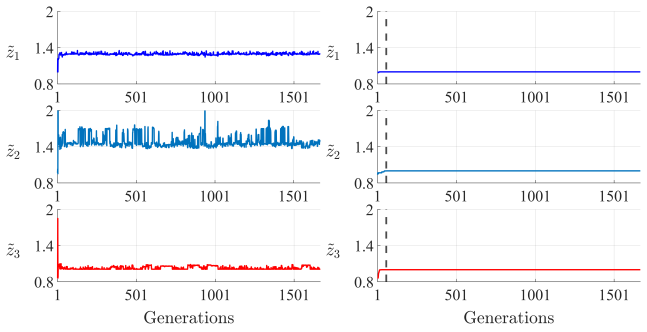
Table 1: Temporal Stability of Nadir Point Estimation (19 of 88 Problems)

				Ideal Point		Nadir Point		
Problem	**Algorithm**	**Final IGD$^+$ (↓)**	**Final HV (↑)**	**%Stable**	**Avg Gen**	**%Stable**	**Avg Gen**	**Verdict**
Two-Objective Problems (N = 120, MaxFE= 200000, MaxGen= 1667)								
BT1	Pl-NSGA-III	9.02e-03 ± 3.91e-03	0.9826 ± 0.0068	98%	1077	22%	1606	Both*
	Py-NSGA-III	9.39e-03 ± 4.31e-03	0.9816 ± 0.0073	99%	1108	79%	1476	
BT6	Pl-NSGA-III	2.18e-01 ± 1.94e-02	0.5495 ± 0.0330	100%	274	73%	1333	Neither†
	Py-NSGA-III	2.17e-01 ± 1.75e-02	0.5503 ± 0.0304	100%	272	17%	1610	
IMOP1	Pl-NSGA-III	1.66e-03 ± 7.35e-05	0.9978 ± 0.0001	100%	128	5%	1632	Both*
	Py-NSGA-III	1.74e-03 ± 9.58e-05	0.9977 ± 0.0001	100%	113	99%	857	
RWA1	Pl-NSGA-III	1.29e-01 ± 1.17e-04	0.9964 ± 0.0000	100%	53	100%	59	Both
	Py-NSGA-III	1.29e-01 ± 1.32e-04	0.9964 ± 0.0000	100%	51	100%	56	
UF1	Pl-NSGA-III	7.11e-02 ± 1.36e-02	0.8446 ± 0.0306	100%	998	44%	1519	Neither
	Py-NSGA-III	7.06e-02 ± 1.19e-02	0.8434 ± 0.0287	100%	1002	89%	1428	
ZDT1	Pl-NSGA-III	2.03e-03 ± 1.84e-05	0.9966 ± 0.0000	100%	183	96%	249	Both*
	Py-NSGA-III	2.03e-03 ± 1.44e-05	0.9966 ± 0.0000	100%	186	100%	460	
ZDT3	Pl-NSGA-III	1.39e-03 ± 8.52e-05	0.9986 ± 0.0001	100%	200	100%	270	Both*
	Py-NSGA-III	1.32e-03 ± 1.03e-04	0.9984 ± 0.0002	100%	195	100%	416	
Three-Objective Problems (N = 120, MaxFE= 200000, MaxGen= 1667)								
DTLZ1	Pl-NSGA-III	4.49e-05 ± 5.38e-05	0.9834 ± 0.0000	100%	4	100%	588	Both
	Py-NSGA-III	4.17e-05 ± 4.48e-05	0.9834 ± 0.0000	100%	4	100%	528	
DTLZ2	Pl-NSGA-III	4.06e-05 ± 1.46e-05	0.9697 ± 0.0000	100%	4	100%	198	Both
	Py-NSGA-III	6.95e-05 ± 4.07e-05	0.9697 ± 0.0000	100%	4	100%	550	
IDTLZ1	Pl-NSGA-III	1.55e-02 ± 6.16e-04	0.8988 ± 0.0031	100%	15	70%	1512	Py-NSGA-III
	Py-NSGA-III	1.56e-02 ± 6.51e-04	0.8983 ± 0.0033	100%	16	97%	1168	
IDTLZ2	Pl-NSGA-III	3.27e-02 ± 2.02e-03	0.9242 ± 0.0033	100%	18	0%	–	Py-NSGA-III
	Py-NSGA-III	3.26e-02 ± 1.74e-03	0.9301 ± 0.0065	100%	19	96%	1186	
IMOP6	Pl-NSGA-III	3.86e-02 ± 6.16e-02	0.9451 ± 0.0918	100%	158	3%	1590	Py-NSGA-III
	Py-NSGA-III	2.90e-02 ± 4.45e-02	0.9602 ± 0.0665	100%	182	100%	739	
MaF1	Pl-NSGA-III	3.62e-02 ± 1.34e-03	0.8858 ± 0.0039	99%	348	46%	1469	Py-NSGA-III
	Py-NSGA-III	3.44e-02 ± 2.27e-03	0.8903 ± 0.0057	99%	341	85%	1121	
−DTLZ1	Pl-NSGA-III	1.90e+01 ± 8.29e-01	0.8903 ± 0.0049	100%	307	0%	–	Py-NSGA-III
	Py-NSGA-III	1.83e+01 ± 1.10e+00	0.8930 ± 0.0050	100%	238	100%	5	
−DTLZ2	Pl-NSGA-III	1.12e-01 ± 5.60e-03	0.9253 ± 0.0032	100%	364	0%	–	Py-NSGA-III
	Py-NSGA-III	1.14e-01 ± 5.55e-03	0.9297 ± 0.0051	100%	136	100%	17	
−WFG1	Pl-NSGA-III	2.15e-01 ± 8.98e-02	0.7625 ± 0.0778	100%	55	25%	1598	Py-NSGA-III
	Py-NSGA-III	1.54e-01 ± 8.85e-02	0.8316 ± 0.0885	100%	56	100%	186	
−WFG2	Pl-NSGA-III	5.32e-02 ± 3.30e-03	0.9313 ± 0.0045	100%	211	0%	–	Py-NSGA-III
	Py-NSGA-III	4.31e-02 ± 1.04e-02	0.9416 ± 0.0108	100%	174	0%	–	
WFG1	Pl-NSGA-III	7.23e-03 ± 1.56e-03	0.9901 ± 0.0007	100%	303	7%	1625	Both*
	Py-NSGA-III	1.25e-02 ± 5.56e-03	0.9901 ± 0.0007	100%	297	99%	742	
WFG2	Pl-NSGA-III	2.27e-03 ± 4.83e-04	0.9929 ± 0.0003	100%	422	89%	1443	Pl-NSGA-III†
	Py-NSGA-III	3.44e-03 ± 1.64e-03	0.9930 ± 0.0003	100%	434	58%	1536	

4.2 Results and Discussion

Table 1 summarizes selected results (full results in Supplementary Materials). Minus-DTLZ and Minus-WFG are abbreviated as −DTLZ and −WFG.

Verdicts marked *. On several two-objective problems, Py-NSGA-III achieves temporal stability for nadir estimation but exhibits non-negligible bias from the true nadir point. We attribute this to a safeguard in Py-NSGA-III: objective values within 10^{-3} of the ideal point are replaced by the ideal point values prior to extreme point selection, which can induce systematic bias.



(a) Pl-NSGA-III. (b) Py-NSGA-III.

Figure 4: Normalized estimated nadir point $\tilde{\mathbf{z}}^{(t)}$ for both algorithms on RWA3 (same initial population). Dotted line: $t^* = 56$.

Table 2: Summary across 100 Problem Instances

Comparison	IGD$^+$	HV	Nadir %Stable	Nadir Avg Gen	Verdict
Pl-NSGA-III wins	38	28	5	14	2
Py-NSGA-III wins	57	52	69	70	36
Ties	5	20	12	2	37
Incomparable	–	–	14	14	25

Verdicts marked †. Among 100 instances, Pl-NSGA-III hardly outperforms Py-NSGA-III in stability. On BT6, both algorithms struggle, but Pl-NSGA-III achieves temporal stability on 73% of runs versus only 17% for Py-NSGA-III. Since BT6 is heavily biased, all solutions have objectives within 10^{-3} of the estimated ideal point, causing Py-NSGA-III's safeguard to collapse all extreme points to $(0, 0)^\top$ in the shifted space. On WFG2, the 10^{-3} safeguard similarly destabilizes estimation because the highly convex landscape near boundary solutions maps a continuous region to a single point, causing instability in extreme point selection.

Table 2 summarizes across all 100 instances. Py-NSGA-III outperforms Pl-NSGA-III in IGD$^+$ (57 vs. 38), HV (52 vs. 28), and nadir point stability (69 vs. 5), leading to an overall verdict preference of 36 vs. 2 for Py-NSGA-III.

5 Conclusion

We compared NSGA-III implementations in PlatEMO and pymoo across 100 problem instances. Using temporal stability metrics for nadir point estimation, we found that Py-NSGA-III outperforms Pl-NSGA-III on most problems (69 vs. 5 in nadir stability), though Py-NSGA-III's 10^{-3} safeguard destabilizes a few cases. Researchers should be aware that these two implementations may yield different results. Future work will analyze the sources of instability and develop improved variants.

References

[1] Julian Blank and Kalyanmoy Deb. 2020. pymoo: multi-objective optimization in Python. *IEEE Access* 8 (2020), 89497–89509. doi:10.1109/ACCESS.2020.2990567

[2] Julian Blank, Kalyanmoy Deb, and Proteek Chandan Roy. 2019. Investigating the normalization procedure of NSGA-III. In *Proceedings of the 10th International Conference on Evolutionary Multi-Criterion Optimization (EMO 2019)*. Springer, Cham, 229–240. doi:10.1007/978-3-030-12598-1_19

[3] Ran Cheng, Miqing Li, Ye Tian, Xingyi Zhang, Shengxiang Yang, Yaochu Jin, and Xin Yao. 2017. A benchmark test suite for evolutionary many-objective optimization. *Complex & Intelligent Systems* 3, 1 (March 2017), 67–81. doi:10.1007/s40747-017-0039-7

[4] Kalyanmoy Deb and Himanshu Jain. 2014. An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part I: solving problems with box constraints. *IEEE Transactions on Evolutionary Computation* 18, 4 (2014), 577–601. doi:10.1109/TEVC.2013.2281535

[5] Kalyanmoy Deb, Lothar Thiele, Marco Laumanns, and Eckart Zitzler. 2002. Scalable multi-objective optimization test problems. In *Proceedings of the 2002 Congress on Evolutionary Computation. CEC'02 (Cat. No.02TH8600)* (Honolulu, HI, USA) *(CEC '02, Vol. 1)*. IEEE, 825–830. doi:10.1109/CEC.2002.1007032

[6] Cheng Gong, Yang Nan, Lie Meng Pang, Hisao Ishibuchi, and Qingfu Zhang. 2024. Performance of NSGA-III on multi-objective combinatorial optimization problems heavily depends on its implementations. In *Proceedings of the Genetic and Evolutionary Computation Conference* (Melbourne, VIC, Australia) *(GECCO '24)*. Association for Computing Machinery, New York, NY, USA, 511–519. doi:10.1145/3638529.3654004

[7] Amir M. Hossain and Hasegawa Takashi. 1989. Nonlinear mixed-discrete structural optimization. *Journal of Structural Engineering-asce* 115, 3 (1989), 626–646. doi:10.1061/(ASCE)0733-9445(1989)115:3(626)

[8] Simon Huband, Luigi Barone, Lyndon While, and Phil Hingston. 2005. A scalable multi-objective test problem toolkit. In *Evolutionary Multi-Criterion Optimization* (Guanajuato, Mexico) *(EMO '05)*. Springer Berlin Heidelberg, Berlin, Heidelberg, 280–295. doi:10.1007/978-3-540-31880-4_20

[9] Hisao Ishibuchi, Yu Setoguchi, Hiroyuki Masuda, and Yusuke Nojima. 2017. Performance of decomposition-based many-objective algorithms strongly depends on Pareto front shapes. *IEEE Transactions on Evolutionary Computation* 21, 2 (2017), 169–190. doi:10.1109/TEVC.2016.2587749

[10] Himanshu Jain and Kalyanmoy Deb. 2014. An evolutionary many-objective optimization algorithm using reference-point based nondominated sorting approach, part II: handling constraints and extending to an adaptive approach. *IEEE Transactions on Evolutionary Computation* 18, 4 (2014), 602–622. doi:10.1109/TEVC.2013.2281534

[11] Hui Li, Qingfu Zhang, and Jingda Deng. 2017. Biased multiobjective optimization and decomposition algorithm. *IEEE Transactions on Cybernetics* 47, 1 (2017), 52–66. doi:10.1109/TCYB.2015.2507366

[12] Ye Tian, Ran Cheng, Xingyi Zhang, and Yaochu Jin. 2017. PlatEMO: a MATLAB platform for evolutionary multi-objective optimization [educational forum]. *IEEE Computational Intelligence Magazine* 12, 4 (2017), 73–87. doi:10.1109/MCI.2017.2742868

[13] Ye Tian, Ran Cheng, Xingyi Zhang, Miqing Li, and Yaochu Jin. 2019. Diversity assessment of multi-objective evolutionary algorithms: performance metric and benchmark problems [research frontier]. *IEEE Computational Intelligence Magazine* 14, 3 (2019), 61–74. doi:10.1109/MCI.2019.2919398

[14] Rémy Viennet, Christian Fonteix, and Ivan Marc. 1996. Multicriteria optimization using a genetic algorithm for determining a Pareto set. *International Journal of Systems Science* 27, 2 (1996), 255–260. doi:10.1080/00207729608929211

[15] Qingfu Zhang, Aimin Zhou, Shi-Zheng Zhao, Ponnuthurai Nagaratnam Suganthan, Wudong Liu, and Santosh Tiwari. 2009. *Multiobjective optimization test instances for the CEC 2009 special session and competition.* Technical Report CES-487. School of CS & EE, University of Essex, Colchester, UK.

[16] Aimin Zhou, Bo-Yang Qu, Hui Li, Shi-Zheng Zhao, Ponnuthurai Nagaratnam Suganthan, and Qingfu Zhang. 2011. Multiobjective evolutionary algorithms: A survey of the state of the art. *Swarm and Evolutionary Computation* 1, 1 (2011), 32–49. doi:10.1016/j.swevo.2011.03.001

[17] Eckart Zitzler, Kalyanmoy Deb, and Lothar Thiele. 2000. Comparison of multiobjective evolutionary algorithms: empirical results. *Evolutionary Computation* 8, 2 (2000), 173–195. doi:10.1162/106365600568202

Acknowledgments

This work was supported by National Natural Science Foundation of China (Grant No. 62376115), and Guangdong Provincial Key Laboratory (Grant No. 2020B121201001). The authors acknowledge the use of generative AI (Claude, Anthropic and Gemini, Google) for assistance with code implementation and manuscript preparation.